Chapter 2 worked solutions

DS02-E01 • CSR and transpose

CSR is `row_ptr=[0,1,3,4]`, `col_idx=[1,0,3,1]`, and `values=[5,-2,7,4]`. Original column counts are `[1,2,0,1]`, with starts `[0,1,3,3]`. In original entry order, output positions are 1,0,3,2. The transposed COO is `(0,1,-2),(1,0,5),(1,2,4),(3,1,7)`, with dimensions 4×3. Its column counts `[1,2,1]` and starts `[0,1,3]` transpose it back to the original dimensions and sorted triples. Check dimensions, coordinates, and values, not just nnz.

DS02-E02 • Four-dimensional addresses and capacity

The element offset is $(((i_0D_1+i_1)D_2+i_2)D_3+i_3)$, or $i_0D_1D_2D_3+i_1D_2D_3+i_2D_3+i_3$. Multiply by w for the byte offset. Require $0 \le i_j < D_j$. Both the dimension product and its product with w must fit the allocation API's size type and container or platform limits. Valid indices alone do not exclude intermediate multiplication overflow. Check bounds before multiplying.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DS02-E03 • KMP trace

The prefix function is $\pi = [0, 1, 2, 3, 4, 0]$. Here q is the number of matched pattern symbols. Text indices start at zero. Each row processes the same current symbol until it matches or q reaches zero; the text index never moves backward.

Text index i	T[i]	q before	Mismatch fallback	q after
0	a	0	None	1
1	a	1	None	2
2	a	2	None	3
3	a	3	None	4
4	a	4	None	5
5	a	5	P[5]=b mismatches; q=π[4]=4; match the same a against P[4]	5
6	a	5	5→4, then match P[4]	5
7	a	5	5→4, then match P[4]	5
8	a	5	5→4, then match P[4]	5
9	b	5	Matches P[5]=b; no fallback	6

At $i = 9$, $q = 6 = |P|$, so the first match starts at $9 - 6 + 1 = 4$. Naive matching compares five a symbols again at each start 0, 1, 2, and 3, then mismatches where it expects b. Start 4 takes six comparisons to succeed. The total is $4 \cdot 6 + 6 = 30$ symbol comparisons. On longer texts of this form, repeated work can reach $\Theta(NM)$. Every KMP fallback decreases q, and forward extensions pay for all increases, giving $O(N + M)$ time including prefix construction.

DS02-E04 • Does CSR always save space?

COO stores two indices and one value per entry, giving `16k` bytes. CSR stores a column and value per entry plus row pointers, giving `12k+4(R+1)` bytes. CSR is smaller exactly when `R+1<k`. For the given values COO uses 1,600 bytes and CSR 4,001,204 bytes. Row pointers dominate. CSR may still help row operations, but it does not always save space. Headers, alignment, compression, and alternate index widths are excluded, so these formulas are not universal exact allocation sizes.

DS02-E05

The element offset is $2 \cdot 4 + 1 = 9$, the byte offset is $9 \cdot 8 = 72$, and the address is $b + 72$. Advancing the inner column index moves 8 bytes; advancing the inner row index moves $4 \cdot 8 = 32$ bytes. Both visit 12 elements. Cache lines, starting state, compiler, and hardware affect elapsed time, so strides alone do not determine a speed ratio.

DS02-E06

`row_ptr=[0,0,1,1,2]`, `col_idx=[2,0]`, and `values=[5,-1]`. The row intervals are [0,0), [0,1), [1,1), [1,2); rows 0 and 2 are empty. The invalid pointer ends at 3 although $k = 2$, violating `row_ptr[R]=k` and extending the last interval beyond the data.

DS02-E07

The entries at (0,1) sum to zero and are removed; those at (2,0) sum to 3. The canonical result is `(2,0,3)`, and its transpose is `(0,2,3)`. The input repeats coordinates and contains a pair that cancels. Normalization is needed before the merge procedure requiring unique coordinates.

DS02-E08

`reserve(10)` leaves size at 2. Since the original capacity is 2 and at least 10 is requested, reallocation occurs and p is invalidated; dereferencing it is invalid. Index 2 is still outside `[0,size)`, so `v[2]` is not a valid element. Increasing capacity does not construct elements or increase size.

DS02-E09

`A` and `B` use one byte each and 你 uses three, for five bytes. `B` starts at byte offset 4 and code-point offset 2. A grapheme cluster may contain multiple code points, while one UTF-8 code point may occupy multiple bytes, so neither offset is generally a grapheme index.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

DS02-E10

Row-major element offset is $((1 \cdot 3) + 2) \cdot 4 + 1 = 21$, giving byte offset $21 \cdot 4 = 84$; the last index changes fastest. Column-major gives $1 + 2(2 + 3 \cdot 1) = 11$, or $11 \cdot 4 = 44$ bytes; the first index changes fastest. Both assume one contiguous block, valid indices, and no arithmetic overflow.

DS02-E11

The prefix function of `aba` is `[0,0,1]`. The first match ends at index 2 and starts at 0. Set $q = \pi[2] = 1$ to retain its final `a`; reading `b` at 3 and `a` at 4 gives a second match starting at 2. Clearing q to zero makes the `b` at index 3 fail against the first `a`, losing the overlapping occurrence at 2.

DS02-E12

The transpose has dimensions 4×0 and no entries. Its four empty rows require CSR `row_ptr=[0,0,0,0,0]`, with empty column and value arrays. Dimensions still determine the number of rows and columns; CSR needs $R + 1 = 5$ row boundaries to represent those rows. With $C = 4, k = 0$, counting and prefix scanning still cost $\Theta(C)$.

References and attribution

Lin Cing, handwritten data structures notes; C++ draft vector.overview and vector.capacity; Knuth, Morris, Pratt, Fast Pattern Matching in Strings (1977).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study